BMS PLATFORM · FIELD REFERENCE

Onboarding a new company

From "we signed a customer" to "their readings are on the dashboard" — who you talk to, what you create, and what you hand over before anyone drives to site.

How it works

You are the middle step. The IoT team knows the **equipment**; you own the **configuration**; they cannot start until you give them values that do not exist until you create them.

```
1. THEY GIVE YOU          2. YOU CREATE              3. YOU GIVE BACK
   ----------------          -------------              ---------------
   Company name       ──→    Tenant (slug)       ──→    Broker host + port
   Site + address     ──→    Site (code)         ──→    Username + password
   How many devices   ──→    Device records      ──→    Topic per device
   Equipment per unit        Device type               CA certificate
   Vendor sample payload ──→ Payload mapping     ──→    Expected payload
   Alert thresholds          Alert rules               Which code = which machine
```

Why it has to be a round trip

The topic `iot/abc-manufacturing/bangalore-factory/vib-comp-3/telemetry` is three words **you invent** when you create the tenant, the site and the device. Nobody can know it in advance, and the device cannot discover it. That is the whole reason the IoT team has to wait for you.

The topic, decoded

```
iot / {tenant.Slug}        / {site.Code}         / {device.DeviceCode} / telemetry
      "abc-manufacturing"    "bangalore-factory"   "vib-comp-3"
      ↑ you choose           ↑ you choose          ↑ you choose
```

Last segment varies: `telemetry` for readings, `heartbeat` for liveness, `commands/ack` for command responses. All segments lowercase — MQTT topics are case-sensitive.

Step 1 — Sitting with the IoT team and the customer

Three different people, three sets of questions. Get them on one call if you can.

The maintenance / facility manager

- Which equipment exactly? Make, model, where it sits
- What do you want to *know*? ("Is the compressor vibrating abnormally" — not "give me data")
- What number means something is wrong? → becomes your alert threshold
- **Who receives the alert, and how?** Email, Telegram, CMMS work order
- What does normal look like today? You need this so thresholds do not cry wolf

Their IT person — the one everyone forgets

- Can a device on the factory network reach your broker on port 8883 outbound?
- Firewall or proxy in the way? Do they need to raise a rule?
- Wired or wifi? If wifi, you need SSID and password on install day

> **Ask this a week ahead, in writing**
> A device that cannot reach the broker looks exactly like a broken device. This single unanswered question is the commonest cause of a wasted site visit.

The device vendor

- **A real sample payload** — actual JSON from an actual device. This is the one thing you cannot proceed without
- Protocol and port: MQTT, or TCP?
- Does it support TLS, and will it accept your CA?
- Can broker host, topic, username and password be configured on the device?

> **Leave the meeting with**
> Equipment list · alert thresholds · alert recipients · the network answer · and a promised sample payload. Without the payload you cannot build the mapping, and without the mapping nothing decodes.

Step 2 — What you create, in this order

Each depends on the one before it.

1 **Tenant**

Name "ABC Manufacturing", slug `abc-manufacturing`. Lowercase, no spaces.

The slug is the MQTT username *and* the first topic segment, and the broker ACL confines the tenant to `iot/{slug}/#`. Changing it later breaks every device at once. Choose it like a primary key.

2 **MQTT credential**

One per **tenant**, not per device. Username is always the slug. **Record the password now.**

3 **Site**

Code `bangalore-factory` — second topic segment. Lowercase.

4 **Device type + parameters**

A class of equipment ("WBGT Sensor"), not one unit. Its parameter list is an **allow-list**: a field arriving under a name not in it is silently dropped.

Set Min/Max wide enough for a real fault. A heat-stress sensor clamped to 18–38 °C cannot report the 45 °C reading you most want to alert on.

5 **Payload mapping**

Vendor's names → your parameter keys. Flat renames use a FieldMap:

```
{"tdb": "dryBulb", "tnwb": "wetBulb", "tg": "globeTemp", "rh": "humidity"}
```

Keys are what the vendor sends; values must match the device type's parameter keys **exactly**. Nested JSON cannot be handled by a flat rename — it needs a MapperKey instead.

6 **Device**

One per physical unit. Code `vib-comp-3` — third topic segment. Pick its type, its mapping, and its transport.

Prove it before the hardware exists

```
mosquitto_pub -h <broker> -p 8883 --cafile ca.crt \
  -u abc-manufacturing -P '<password>' \
  -t 'iot/abc-manufacturing/bangalore-factory/vib-comp-3/telemetry' \
  -m '{"tdb":34.2,"tnwb":26.1,"tg":39.0,"rh":64}'
```

Then check it landed as `Persisted`. If it is `Rejected`, fix it at your desk — not on a factory floor with an electrician waiting.

Step 3 — What you send before they travel

A week ahead — to the customer's IT

```
Devices need outbound TCP to mqtt.yourcompany.com port 8883.
No inbound ports required.
```

Confirmed in writing. Slowest thing to resolve, commonest cause of a second visit.

The day before — to the IoT team

#	WHAT	EXAMPLE
1	**Broker host + port**	`mqtt.yourcompany.com` : `8883` (TLS, never 1883)
2	**Username**	`abc-manufacturing` — same for every ABC device
3	**Password**	**separate channel** — see below
4	**CA certificate**	the .crt file, if you run your own CA
5	**Device list**	equipment ↔ code ↔ topic (table below)
6	**Expected payload**	`{"tdb":34.2,"tnwb":26.1,"tg":39.0,"rh":64}`
7	**Verification + your number**	"call us before you leave site"

Item 5 — the one that matters most

PHYSICAL EQUIPMENT	DEVICE CODE	PUBLISH TO
Compressor 3, Line A	`vib-comp-3`	`iot/abc-manufacturing/bangalore-factory/vib-comp-3/telemetry`
Shop floor, Bay 2	`wbgt-bay-2`	`iot/abc-manufacturing/bangalore-factory/wbgt-bay-2/telemetry`
Press area	`noise-press-1`	`iot/abc-manufacturing/bangalore-factory/noise-press-1/telemetry`

> **Make them confirm location against code before leaving**
>
> Swap two rows by accident and everything still works — data flows, no errors — but the compressor's readings are labelled as the press. You find out weeks later, when an alert makes no sense.

Step 3 — continued: the password

The password — separately

Best	Shared password-manager vault — revocable, auditable, removable when someone leaves
Good	One-time secret link that self-destructs — for an external contractor
Fine	Read it out over the phone. For three devices this is often simplest
Never	Same email as the sheet · a Slack channel with history · a shared spreadsheet

> **It is the company's key, not one sensor's**
>
> Every ABC device logs in as `abc-manufacturing` with the same password — devices are identified by their *topic*, not their login. Rotating it re-credentials every ABC device, which is a site visit, not a button press. Give it to as few people as possible.

Never sent

Dashboard logins · device types · payload mappings · anything about platform internals. Those need judgement about field names that fails silently when wrong.

Step 4 — Verify, in this order

Each check rules out the one before it. Do these while the installer is still on site.

a. Is it connecting and publishing?

```
mosquitto_sub -h <broker> -p 8883 --cafile ca.crt \
  -u abc-manufacturing -P '<password>' \
  -t 'iot/abc-manufacturing/bangalore-factory/#' -v
```

Nothing arriving → network, then credentials, then topic spelling, then TLS trust, in that order.

b. Did we store it, and what did we make of it?

```
SELECT "ReceivedAt", "ProcessingStatus", "ProcessingError"
FROM "RawDeviceMessages"
WHERE "DeviceId" = '<device-id>'
ORDER BY "ReceivedAt" DESC LIMIT 5;
```

The single most useful query in onboarding.

STATUS	MEANING
Persisted	Good — decoded, validated, stored
Rejected — "No recognized telemetry fields present after mapping"	**Most common failure.** Device type and payload mapping do not line up
Rejected — validation message	A value fell outside the parameter's Min/Max
nothing at all	It never reached ingestion — back to (a)

c. Does it reach the dashboard?

Live telemetry should update within seconds and the chart should populate. Then set the alert rules while you still have the customer's attention.

The mistake you will make

Nine times in ten, a device that "isn't working" is a name mismatch. The chain must line up end to end:

```
vendor sends      {"tdb": 34.0}
FieldMap          {"tdb": "dryBulb"}      ← key   = vendor's name
parameter Key      dryBulb                ← value must match this exactly
```

Break one link and that field vanishes silently. Break every link and the message is rejected outright.

> **Nested JSON needs a MapperKey, not a FieldMap**
> A flat rename table cannot reach inside `{"sensor":{"tdb":34}}`. If a vendor sends nested payloads, the mapping needs a registered mapper instead — otherwise every message from that device is rejected.

Reusable order of work

1. **Get a real sample payload first** — it determines steps 4 and 5
2. Network requirement to customer IT, a week ahead, in writing
3. Tenant → MQTT credential → site
4. Device type with parameters taken from that sample
5. Payload mapping whose outputs match those parameter keys
6. Device records
7. Prove it with `mosquitto_pub` before the hardware arrives
8. Send the handover pack; password separately
9. Install, verify live while they are on site
10. Alert rules and recipients, then walk the manager through the dashboard

Half a day the first time for a new equipment model; about an hour once the device type and mapping already exist and can be reused.